

Supplement for the Joint Sinkhorn Auto-Encoder (jSAE) design

In this supplement we detail the design of our jSAE with the goal of learning to sample from a joint distribution $P_{X,Y}$ given a dataset of realizations of the tuple (X, Y) . We refer to the inputs to the jSAE by (X, Y) and its outputs by $(\tilde{X}, \tilde{Y})$. Furthermore, the inputs' marginal distributions are denoted as P_X , P_Y . The generated distributions are denoted as $P_{\tilde{X}}$, $P_{\tilde{Y}}$, and $P_{\tilde{X},\tilde{Y}}$. The latent distributions are denoted as P_Z , for the prescribed prior, and Q_Z , for the aggregated encoded variables distribution.

Architecture components

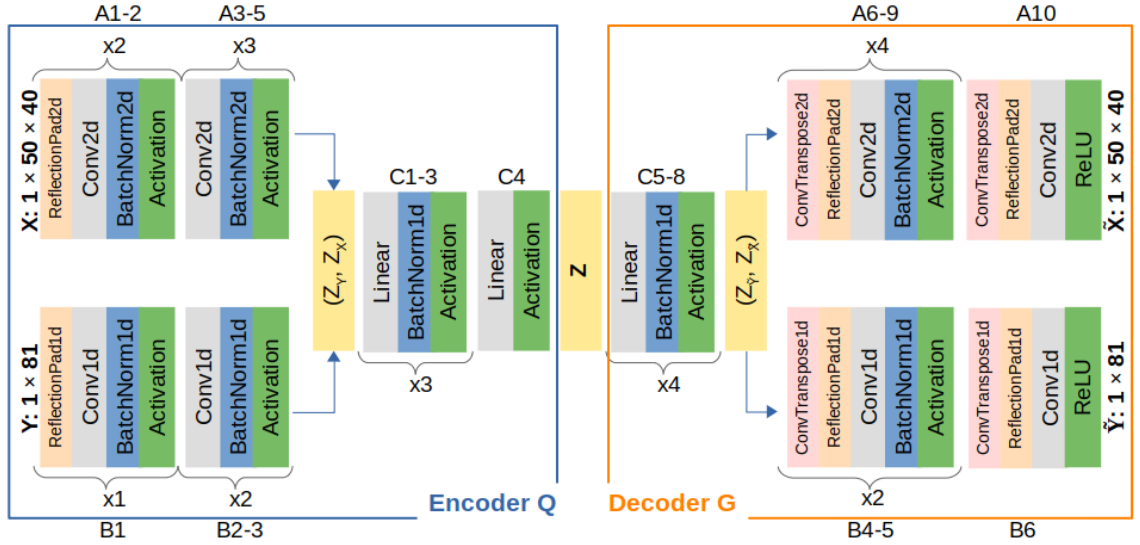


Figure 1: Schematic our Encoder-Decoder jSAE architecture and its components.

The Encoder-Decoder stacks downsampling blocks in the encoder followed by up-sampling blocks in the decoder. A forward pass through the network goes from left to right. We detail the configuration of the blocks following the numbering scheme shown in Figure 1. We represent convolution and transposed convolution function arguments in the following order: number of input channels, number of output channels, kernel size, stride, padding. The bias parameter was activated for all layers and

is not shown. The letter b denotes the batch size. The letter k denotes the latent space dimension. We refer the reader to the Pytorch documentation¹ for details on the layers' arguments.

Encoder - Downsampling

A1: input $X \xrightarrow{(b,1,50,40)}$ ReflectionPad2d(2) $\xrightarrow{(b,1,54,44)}$ Conv2d(1, 64, 5, 2, 0) $\xrightarrow{(b,64,25,20)}$ BatchNorm2d(64) $\xrightarrow{(b,64,25,20)}$ activation function $\xrightarrow{(b,64,25,20)}$ A1 output

A2: A1 output $\xrightarrow{(b,64,25,20)}$ ReflectionPad2d(2) $\xrightarrow{(b,64,29,24)}$ Conv2d(64, 128, 5, 2, 0) $\xrightarrow{(b,128,13,10)}$ BatchNorm2d(128) $\xrightarrow{(b,128,13,10)}$ activation function $\xrightarrow{(b,128,13,10)}$ A2 output

A3: A2 output $\xrightarrow{(b,128,13,10)}$ Conv2d(128, 256, (5,2), 1, 0) $\xrightarrow{(b,256,9,9)}$ BatchNorm2d(256) $\xrightarrow{(b,256,9,9)}$ activation function $\xrightarrow{(b,256,9,9)}$ A3 output

A4: A3 output $\xrightarrow{(b,256,9,9)}$ Conv2d(256, 512, 1, 1, 0) $\xrightarrow{(b,512,9,9)}$ BatchNorm2d(512) $\xrightarrow{(b,512,9,9)}$ activation function $\xrightarrow{(b,512,9,9)}$ A4 output

A5: A4 output $\xrightarrow{(b,512,9,9)}$ Conv2d(512, 1, 1, 1, 0) $\xrightarrow{(b,1,9,9)}$ BatchNorm2d(1) $\xrightarrow{(b,1,9,9)}$ activation function $\xrightarrow{(b,1,9,9)}$ reshape(b, 81) $\xrightarrow{(b,81)}$ A5 output

B1: input $Y \xrightarrow{(b,1,81)}$ ReflectionPad1d(1) $\xrightarrow{(b,1,83)}$ Conv1d(1, 256, 3, 2, 0) $\xrightarrow{(b,256,41)}$ BatchNorm1d(256) $\xrightarrow{(b,256,41)}$ activation function $\xrightarrow{(b,256,41)}$ B1 output

B2: B1 output $\xrightarrow{(b,256,41)}$ Conv1d(256, 512, 1, 1, 0) $\xrightarrow{(b,512,41)}$ BatchNorm1d(512) $\xrightarrow{(b,512,41)}$ activation function $\xrightarrow{(b,512,41)}$ B2 output

B3: B2 output $\xrightarrow{(b,512,41)}$ Conv1d(512, 1, 1, 1, 0) $\xrightarrow{(b,1,41)}$ BatchNorm1d(1) $\xrightarrow{(b,1,41)}$ activation function $\xrightarrow{(b,1,41)}$ reshape(b, 41) $\xrightarrow{(b,41)}$ B3 output

C1: concatenate(A5 output, B3 output) $\xrightarrow{(b,122)}$ Linear(122, 122) $\xrightarrow{(b,122)}$ BatchNorm1d(122) $\xrightarrow{(b,122)}$ activation function $\xrightarrow{(b,122)}$ C1 output

C2: C1 output $\xrightarrow{(b,122)}$ Linear(122, $k \times 3$) $\xrightarrow{(b,k \times 3)}$ BatchNorm1d($k \times 3$) $\xrightarrow{(b,k \times 3)}$ activation function $\xrightarrow{(b,k \times 3)}$ C2 output

C3: C2 output $\xrightarrow{(b,k \times 3)}$ Linear($k \times 3$, $k \times 2$) $\xrightarrow{(b,k \times 2)}$ BatchNorm1d($k \times 2$) $\xrightarrow{(b,k \times 2)}$

¹<https://docs.pytorch.org/docs/2.0/nn.html>

activation function $\xrightarrow{(b,k \times 2)}$ C3 output

C4: C3 output $\xrightarrow{(b,k \times 2)}$ Linear($k \times 2, k$) $\xrightarrow{(b,k)}$ activation function $\xrightarrow{(b,k)}$ C4 output

Decoder - Upsampling

Upsampling blocks are a succession of transposed convolutions which revert the effect of the convolution used during downsampling followed by convolutions to reduce the artifacts that could appear with transposed convolutions (see [1] and related architecture in [2]).

C5: input Z $\xrightarrow{(b,k)}$ Linear($k, k \times 2$) $\xrightarrow{(b,k \times 2)}$ BatchNorm1d($k \times 2$) $\xrightarrow{(b,k \times 2)}$ activation function $\xrightarrow{(b,k \times 2)}$ C5 output

C6: C5 output $\xrightarrow{(b,k \times 2)}$ Linear($k \times 2, k \times 3$) $\xrightarrow{(b,k \times 3)}$ BatchNorm1d($k \times 3$) $\xrightarrow{(b,k \times 3)}$ activation function $\xrightarrow{(b,k \times 3)}$ C6 output

C7: C6 output $\xrightarrow{(b,k \times 3)}$ Linear($k \times 3, 122$) $\xrightarrow{(b,122)}$ BatchNorm1d(122) $\xrightarrow{(b,122)}$ activation function $\xrightarrow{(b,122)}$ C7 output

C8: C7 output $\xrightarrow{(b,122)}$ Linear(122, 122) $\xrightarrow{(b,122)}$ BatchNorm1d(122) $\xrightarrow{(b,122)}$ activation function $\xrightarrow{(b,122)}$ C8 output

A6: first 81 components of C8 output $\xrightarrow{(b,81)}$ reshape(1, 9, 9) $\xrightarrow{(b,1,9,9)}$ ConvTranspose2d(1, 512, 1, 1, 0) $\xrightarrow{(b,512,9,9)}$ ReflectionPad2d(1) $\xrightarrow{(b,512,11,11)}$ Conv2d(512, 512, 3, 1, 0) $\xrightarrow{(b,512,9,9)}$ BatchNorm2d(512) $\xrightarrow{(b,512,9,9)}$ activation function $\xrightarrow{(b,512,9,9)}$ A6 output

A7: A6 output $\xrightarrow{(b,512,9,9)}$ ConvTranspose2d(512, 256, 1, 1, 0) $\xrightarrow{(b,256,9,9)}$ ReflectionPad2d(1) $\xrightarrow{(b,256,11,11)}$ Conv2d(256, 256, 3, 1, 0) $\xrightarrow{(b,256,9,9)}$ BatchNorm2d(256) $\xrightarrow{(b,256,9,9)}$ activation function $\xrightarrow{(b,256,9,9)}$ A7 output

A8: A7 output $\xrightarrow{(b,256,9,9)}$ ConvTranspose2d(256, 128, (5,2), 1, 0) $\xrightarrow{(b,128,13,10)}$ ReflectionPad2d(1) $\xrightarrow{(b,128,15,12)}$ Conv2d(128, 128, 3, 1, 0) $\xrightarrow{(b,128,13,10)}$ BatchNorm2d(128) $\xrightarrow{(b,128,13,10)}$ activation function $\xrightarrow{(b,128,13,10)}$ A8 output

A9: A8 output $\xrightarrow{(b,128,13,10)}$ ConvTranspose2d(128, 128, (5,6), 2, 2) $\xrightarrow{(b,128,25,20)}$ ReflectionPad2d(1) $\xrightarrow{(b,128,27,22)}$ Conv2d(128, 64, 3, 1, 0) $\xrightarrow{(b,64,25,20)}$ BatchNorm2d(64)

$\xrightarrow{(b,64,25,20)}$ activation function $\xrightarrow{(b,64,25,20)}$ A9 output

A10: A9 output $\xrightarrow{(b,64,25,20)}$ ConvTranspose2d(64, 64, 6, 2, 2) $\xrightarrow{(b,64,50,40)}$ ReflectionPad2d(1) $\xrightarrow{(b,64,52,42)}$ Conv2d(64, 1, 3, 1, 0) $\xrightarrow{(b,1,50,40)}$ ReLU() $\xrightarrow{(b,1,50,40)}$ **output** $\tilde{X}$

B4: last 41 components of C8 output $\xrightarrow{(b,41)}$ reshape(1, 41) $\xrightarrow{(b,1,41)}$ ConvTranspose1d(1, 512, 1, 1, 0) $\xrightarrow{(b,512,41)}$ ReflectionPad1d(1) $\xrightarrow{(b,512,43)}$ Conv1d(512, 512, 3, 1, 0) $\xrightarrow{(b,512,41)}$ BatchNorm1d(512) $\xrightarrow{(b,512,41)}$ activation function $\xrightarrow{(b,512,41)}$ B4 output

B5: B4 output $\xrightarrow{(b,512,41)}$ ConvTranspose1d(512, 256, 1, 1, 0) $\xrightarrow{(b,256,41)}$ ReflectionPad1d(1) $\xrightarrow{(b,256,43)}$ Conv1d(256, 256, 3, 1, 0) $\xrightarrow{(b,256,41)}$ BatchNorm1d(256) $\xrightarrow{(b,256,41)}$ activation function $\xrightarrow{(b,256,41)}$ B5 output

B6: B5 output $\xrightarrow{(b,256,41)}$ ConvTranspose1d(256, 1, 3, 2, 1) $\xrightarrow{(b,1,81)}$ ReflectionPad1d(1) $\xrightarrow{(b,1,83)}$ Conv1d(1, 1, 3, 1, 0) $\xrightarrow{(b,1,81)}$ ReLU() $\xrightarrow{(b,1,81)}$ **output** $\tilde{Y}$

Hyperparameters selection

Our hyperparameters’ validation process aims to enable the presented network structure to work in practice. The validation process requires training the network with different hyperparameters configurations on the training set followed by an estimation of $\mathcal{L}_{jSAE}$, and a set of diagnostic metrics, on the validation set. The configuration that best improves these validation estimates is retained.

We define three groups of hyperparameters to tune: “Architecture”, “Loss function”, and “Training routine”. We summarize them along the tested values in Table 1.

Table 1: Summary list of hyperparameters along their categories and their tested values. Default values are marked by *.

Category	Hyperparameter	Values
Architecture	Encoder smoothness constrained	None* or Spectral norm
Architecture	Activation functions	PReLU (encoder)* + Constr. PReLU (decoder)* or Constr. β -Swish (encoder-decoder)
Architecture	Latent space dimension: k	{10, 15, 20, 25, 30*, 35, 40}
Loss function	Latent space distribution: P_Z	Standard normal*, Uniform $(-1, 1)$, Uniform $(-3, 3)$
Loss function	Sinkhorn algorithm parameters: $\epsilon_{sink}; L$	{10, 100*, 1 000};{100}
Loss function	Norm power: q	{1, 2*}
Loss function	Batch size: m	{100, 200*, 300, 400}
Training routine	$\beta_{sink} = \gamma^q$ scheduling	None*, 1/1.4, adaptive ; starting value $\beta_{sink} = 150$
Training routine	Learning rate scheduling	None*, ReduceLROnPlateau or OneCycleLR
Training routine	Reconstruction loss scaling	None*, $1/dim$, adaptive
Training routine	Data normalization	None*, robust Min-Max

We enumerate a total of 81 648 possible combinations². It is uncommon to exhaustively test all of them³. Instead, we fine-tune the hyperparameters in one category at a time, fixing the hyperparameters in other categories to a default (marked by an asterisk in Table 1) or to a previously found best. The categories tuning order is

²Note that in the “Architecture” category, there are only 3 possible combinations between the activation functions and the encoder smoothness instead of 4

³Neural Architecture Search [3] is a recent prolific research domain for “automating” neural architecture design. We did not venture into it during our work because we assessed that the effort to set its requirement (that is, selecting the search space, the search strategy and the performance evaluation strategy) for our problem, running it, and validating it would be equivalent to directly tuning the design process ourselves.

(1) “Training routine”, (2) “Architecture” and (3) “Loss function”. This procedure reduces the number of combinations to evaluate to 147 in total ($54 + 21 + 72$). We further reduce the number of tested combinations by avoiding running configurations of hyperparameters that showed clear deterioration compared to the baseline or the previous best. Finally, we ended up running 21 training runs in the “Training routine” category, 12 in the “Architecture” category and 22 in the “Loss function” category, for a total of 56 training runs. We train with each hyperparameter configuration for a maximum of 2 000 epochs both to limit computational resources and to assess which combinations converge faster. The training size was set to 7 000 examples. After each training epoch, we evaluate the loss function on the validation set, of size 2 000, and use this estimate as the validation metric for that epoch (e.g., used during the learning rate scheduling with *ReduceLROnPlateau* and to find the best epoch). We evaluate the diagnostic metrics for each tested configuration twice, at the epoch where the loss function estimated on the validation set was the lowest and at the end of the training. By opting for this validation strategy, we control for computational resources, in compute time and storage.

On average, each training run lasts for 1.8 hours (± 8 minutes). Each training runs on a separate NVIDIA GeForce RTX 3090 Graphics Processing Unit (GPU) (2 GPU in total). Generating the diagnostic metrics for each configuration after training, at the lowest validation loss epoch and the last epoch, takes 30 minutes on average. The total average compute time for the hyperparameters tuning was 129 hours (5.4 days).

After selecting the configuration that improved the most on diagnostic metrics, we used this configuration to assess the impact of the data set training size and further run training with sizes $\{10\,000, 14\,000\}$.

The following sections detail the hyperparameter choices and the diagnostic metrics used.

Hyperparameters details

Architecture The latent space dimension plays an important role in approaching the generated distribution to the training data distribution. It needs to be set as close as possible to the, unknown, intrinsic data dimension [4, 5]. We test the values $\{10, 15, 20, 25, 30, 35, 40\}$.

A condition set by [6] on the loss function (Equation ??) is for the regularization parameter $\gamma \geq \|G\|_{Lip}$, the Lipschitz constant of the decoder G . Since γ is not known in advance, we opt to control the norm of G to be at maximum 1. We enforce this condition by: (1) applying spectral normalization⁴ to the weights of each layer of the decoder network; (2) using activation functions for which the maximal rate of change on their domain is bound by 1 (that is, for an activation function g and its derivative g' , we must have $\|g'\|_{\infty} \leq 1$). The choice of the activation function also affects the smoothness of the loss. Rectified Linear Unit (ReLU), one of the simplest and most widely used activation functions, adds discontinuities and irregularities to the loss function due to its discontinuity at the origin. It also contributes to the problem of vanishing gradients, since it only activates for input values larger than 0 and is at 0 for negative input values (see Section). As the network grows in depth, the vanishing gradient causes the learning to stall, which requires increased attention to inputs of activation functions (e.g., via skip connections) to ensure that they remain in the zone where the derivative is nonzero, where inputs are informative. A Parametric Rectified Linear Unit (PReLU) activation helps to maintain the gradient flow by allowing to tune a parameter that regulates the slope of the linearity in the negative part of the domain. This parameter can be learned along the weights of the network. PReLU does not solve the discontinuity at the origin. Smoother choices like β -Swish provided similar or better performance in benchmark problems in very deep neural networks [7, 8]. β -Swish interpolates between a purely linear regime and ReLU by

⁴https://pytorch.org/docs/2.0/generated/torch.nn.utils.spectral_norm.html

tuning its parameter β along the weights of the network. We tested the performance of two activation functions: PReLU in the encoder with a constrained PReLU in the decoder versus a constrained β -Swish in both encoder and decoder. The constrained PReLU in the decoder ensures the condition $\|g'\|_\infty \leq 1$. The constrained β -Swish ensures that the values of β do not enter the linear regime (see Section for details on these activation functions and how we implement the constraints). By choosing parametrized activation functions, we increase the flexibility of the network by allowing the optimization algorithm to tune these parameters.

Finally, there are no explicit smoothness requirements on the encoder. However, we compare the unconstrained case with the constrained smoothness case, where we control the encoder norm by applying spectral normalization. When using PReLU in the encoder, we use the constrained or unconstrained versions as necessary to ensure the smoothness constraint.

Activation functions

ReLU The Rectified Linear Unit (ReLU) is defined as ⁵: $f(x) = \max(0, x)$.

Illustrated in Figure 2, this function is linear for the positive part of its domain and 0 otherwise. The derivative of ReLU is 1 for $x \geq 0$ and 0 otherwise, leading to $\|f'(x)\|_\infty = 1$.

PReLU The Parametric Rectified Linear Unit (PReLU) is defined as ⁶: $f(x) = \max(0, x) + a \cdot \min(0, x)$, where a is a tunable parameter. Unlike ReLU, this function has a slope different from 0 for $x < 0$ (Figure 2). The derivative of this function is 1 for $x \geq 0$ and a otherwise, hence $\|f'(x)\|_\infty = \max(|a|, 1)$.

Constrained PReLU We constrain PReLU to ensure that $\|f'(x)\|_\infty$ is at most

⁵<https://pytorch.org/docs/2.0/generated/torch.nn.ReLU.html?highlight=relu#torch.nn.ReLU>

⁶<https://pytorch.org/docs/2.0/generated/torch.nn.PReLU.html?highlight=prelu#torch.nn.PReLU>

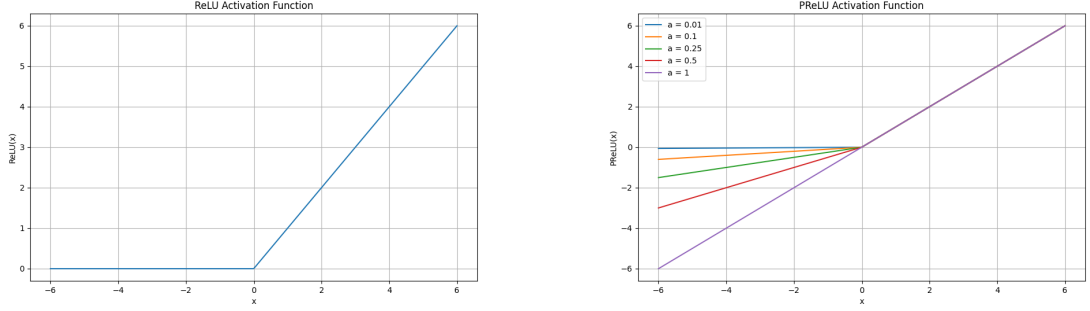


Figure 2: ReLU function (left) and PReLU function (right)

1. For this, we restrict the values of the parameter $|a|$ to be at most 1 (that is, we clamp the value of a to remain between -1 and 1).

β -Swish The β -Swish activation function is defined as follows [7]:

$$f(x) = x \cdot \sigma(\beta x) \text{ with } \sigma(z) = \frac{1}{1 + \exp(-z)}, \text{ with } \beta \text{ a tunable parameter.}$$

Its derivative is defined as follows [7]: $f'(x) = \beta f(x) + \sigma(\beta x)(1 - \beta f(x))$.

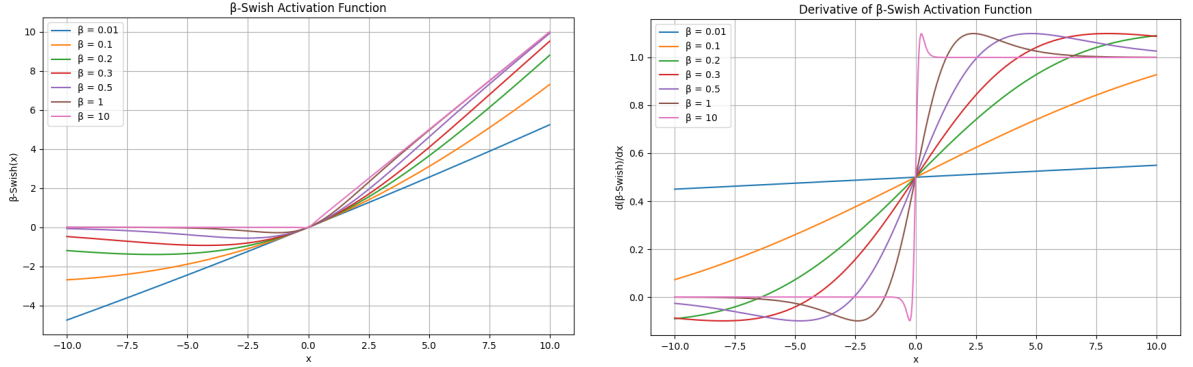


Figure 3: β -Swish function (left) and its derivative (right) for different values of β

Illustrated in Figure 3, β -Swish interpolates between a linear function ($\beta = 0$) and ReLU ($\beta = \infty$). The derivative of β -Swish attains a slight maximum above 1 and then stabilizes at 1, except for linear regimes. The speed at which 1 is reached by the derivative is modulated by β : the higher the value, the faster the saturation.

Constrained β -Swish For most values of β we have $\|f'(x)\|_\infty \approx 1$ (around 1.1), except for small values of β (i.e., in the linear regime). In general, linear functions are not suitable as activation functions in neural networks since they lead to approximating linear functions reducing the flexibility of the network to approximate more

complex maps. Since β is learned during training, we constrain its value to avoid selecting small values that make the function linear. Figure 3 shows that starting from $\beta = 0.1$ there is a clear departure from linearity and its derivative still saturates around 1.1 (see Figure 4 for an illustration on a larger interval). We apply this constraint by forcing the training procedure to set $\beta \geq 0.1$ (that is, we clamp the value of the β parameter from below).

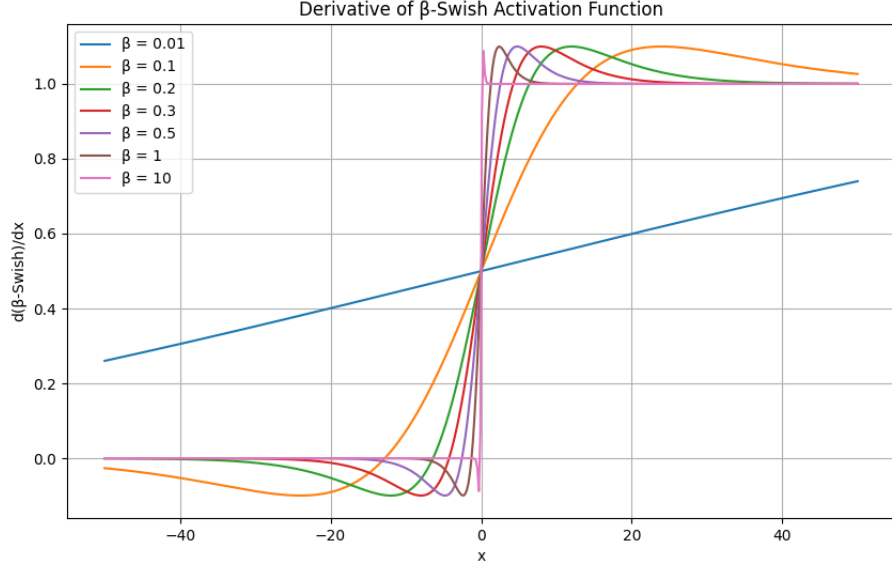


Figure 4: β -Swish function derivative at higher resolution

Training routine Minimizing the loss function $\mathcal{L}_{SAE}$ with an algorithm like SGD requires some adaptations. The purpose of those adaptations is to balance the components of the loss function such that they all contribute in comparable proportions to the learning of the network’s weights.

In encoder-decoder architectures, it is necessary to balance the contribution of the reconstruction error part and the latent distributions’ divergence part. Since the reconstruction error is usually a multiple of magnitudes higher than the latent divergence, the risk is that network training would mainly focus on minimizing the reconstruction part and would not distribute the embeddings in the latent space following P_Z . It becomes important to set the parameter $\beta_{sink} = \gamma^p$ high enough to create this balance. A common technique in such situations is to start with a very large value for this parameter, to force the network to focus on minimizing the latent divergence early on, and then reduce this parameter during training via scheduling (e.g., by dividing β_{sink} by a fixed integer every set number of epochs, to the minimum value of $\sqrt[p]{\beta_{sink}} = 1$) to increase the focus on minimizing the reconstruction error as training continues. We test whether dividing β_{sink} by 1.4 every 100 epochs improves on the quality of the learned outputs. We also test dynamic adaptation of β_{sink} in each training epoch. In this adaptive mode, we set β_{sink} to the result of the division between the average reconstruction error by the average latent divergence, where those averages are computed on the batches of the previous training epoch⁷. This could avoid us finding the best scheduling rate for β_{sink} . Other parameters of the Sinkhorn algorithm are discussed in the “Loss estimation” group.

Another imbalance in the reconstruction part could be aggravated by differences in scales between the two variables X and Y . A common technique to reduce this imbalance is to normalize the training data and bring all variables to comparable scales avoiding the risk of tricking SGD in focusing on learning more from one variable than

⁷We use the training data for this estimation and not the validation data to avoid contamination and accidentally fitting the validation data.

the other. We evaluated whether such normalization could have a positive impact on network training. We namely focus on one form of robust min-max⁸ normalization, where instead of taking the minimum and maximum, we compute a lower (e.g., 1st percentile) and upper percentile (e.g., 99th percentile) for each dimension of the data and use those percentiles in place of minimum and maximum values in normalization. With this normalization, most of the values in the data will fall in the range $[0, 1]$. This robust version reduces the influence of outliers, if they exist. We avoid standardization as it can distort the original data distribution if it is not itself Gaussian (e.g., this would be the case when $F(\cdot)$ is non-linear).

The difference in dimensions between X and Y also leads to an imbalance in the reconstruction part. We tested whether scaling each reconstruction error by its corresponding space dimension (that is, divide the reconstruction error of X by 2 000 and the reconstruction error of Y by 80) could also influence the training quality. We also tested an adaptive approach to balance the reconstructions of X and Y by inflating the reconstruction of Y by a factor that is set to the division of the averages of the reconstruction of X by the reconstruction of Y evaluated from the previous training epoch, similar to the adaptive technique for β_{sink} .

Finally, an important hyperparameter to assess is the learning rate of SGD, the algorithm’s step size in the loss function’s gradient descent direction. The loss, as a function of the neural network weights, is a high-dimensional function, the convexity and smoothness of which depend on the hyperparameter choices (e.g., convexity depends on the strength of the regularization with the latent distributions divergence, smoothness depends on the choice of activation functions in the network layers). The loss functions of neural networks have multiple local minima and saddle points⁹. In general, during weights’ optimization, the aim is to explore as much as possible the

⁸For a data point x , a min-max normalization computes: $x_{normalized} = \frac{x - \min}{\max - \min}$. If x is multidimensional, a min and a max is computed for each dimension and normalization is applied per dimension.

⁹Illustrations of loss landscapes of famous architectures: <https://losslandscape.com>

weights’ space in order to uncover regions where the loss is at its minimum while at the same time controlling for training duration. The learning rate directly influences the exploration capabilities of the gradient descent optimization algorithms. Large learning rates move the algorithm faster towards promising regions of the parameters space with the increased risk to overshoot regions around an adequate minimum and diverge to other regions in the space. Very small learning rates slow convergence of the training and potentially hinder the ability of the algorithm to leave inefficient local minima or cross over saddle points. To account for these risks, learning rate scheduling is commonly used to adapt the learning rate as training progresses, given a predefined strategy. We do this at two levels: (1) we use Adam¹⁰ as the base version of gradient descent algorithm, which automatically applies adaptations to learning rates for each weight in the network as the training progresses; (2) we verify whether learning rate scheduling could positively improve our network training. In the context of Adam, scheduling strategies would be applied on the global learning rate of the algorithm. We test two learning rate scheduling strategies: *ReduceLROnPlateau*¹¹ where the learning rate is reduced when a validation metric plateaus, and *OneCycleLR*¹² where the learning rate is modified during training such that it is first increased and then decreased with a cycling schedule on a range of values. We compare these two strategies against no learning rate scheduling and starting from the default configuration of Adam (lr=0.001, betas=(0.9, 0.999)).

Loss estimation The loss estimation is influenced by the power of the norms used in the reconstruction loss and the Sinkhorn divergence estimation, as well as the latent space distribution P_Z . Theoretical results for the convergence of the Sinkhorn divergence estimations are only available in specific settings (see [9]), namely, in

¹⁰<https://docs.pytorch.org/docs/2.0/generated/torch.optim.Adam.html>

¹¹https://docs.pytorch.org/docs/2.0/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html

¹²https://docs.pytorch.org/docs/2.0/generated/torch.optim.lr_scheduler.OneCycleLR.html

Uniform and Gaussian latent spaces and with degrees 1 and 2 of the $||.||_q^q$ cost on those spaces. We restrict our choices to these options. Furthermore, we fix the number of iterations of the Sinkhorn algorithm to the value of 100 which is considered a relatively large value [10]. By choosing this value, we sacrifice some training time but we provide more iterations for the Sinkhorn algorithm to converge. This compromise allows us to avoid finetuning this parameter and to focus only on tuning the regularization level $\epsilon_{sink} \in \{10, 100, 1\,000\}$, values previously tested in [10].

Finally, the batch size m influences the accuracy of the estimation of the loss function and, more importantly, the estimation of its latent divergence part. However, very high batch sizes could cause overfitting of training data, while low batch sizes would slow down convergence. We test with increasing values in $\{100, 200, 300, 400\}$.

Diagnostic metrics

The evaluation of the loss function on the validation set is the main metric to assess during a training run with each hyperparameter configuration and to compare against its evaluation on the training set. However, this metric is not adequate to compare between configurations that impact the scale of the loss (e.g., when increasing the latent space dimension, the loss estimates increase without necessarily meaning an equivalent deterioration).

Instead of normalizing the validation loss estimates, we opt to quantify the performance of a trained network on three conditions necessary for the success of the inference step: (C1) the generated distributions $P_{\hat{X}}$ and $P_{\hat{Y}}$ are as close as possible to the distributions P_X and P_Y , respectively; (C2) the forward solver is approximated as close as possible by the network; and (C3) the encoding latent space distribution Q_Z is as close as possible to the prescribed P_Z . We summarize the main metrics we used for the comparison in Table 2. A summary of the estimated metrics on a representative subset of configurations is provided in Table ??.

In the following discussion, we refer to the output of the jSAE by (X_{recon}, Y_{recon}) when looking at the network's reconstruction of its inputs (X, Y) . We refer to the network's randomly generated output by $(\tilde{X}, \tilde{Y})$, the transformation by the decoder of a randomly sampled latent vector from P_Z .

Table 2: Summary of conditions and their corresponding diagnostic metrics.

Conditions	Metrics
C1 - P_X and P_Y distributions	- Reconstruction errors (RMSE) for X and Y - kernel two-sample tests: P_X vs $P_{\tilde{X}}$, and P_Y vs $P_{\tilde{Y}}$ - $\widehat{MMD}_b^2(P_X, P_{\tilde{X}})$, $\widehat{MMD}_b^2(P_Y, P_{\tilde{Y}})$
C2 - Forward approximation	- Resimulation errors (RMSE) of: - $\tilde{Y}$ vs $F(\tilde{X})$ - Y_{recon} vs $F(X_{recon})$
C3 - Latent distributions	- kernel two-sample test: P_Z vs Q_Z - $\widehat{MMD}_b^2(P_Z, Q_Z)$

C1 - P_X and P_Y distributions learning We evaluate the reconstruction error of a trained network by computing RMSE between data inputted into the encoder and its reconstruction out of the decoder, for both variables. We evaluate this metric on both the validation and the training data with samples of size 2 000 for each. We summarize the results by the median [2.5th quantile - 97.5th quantile] estimated on these samples. RMSE values indicate component-wise errors/differences between the compared examples. In the context of reconstruction, the ideal value would be 0, perfect reconstructions. A more realistic reference could be to maintain reconstruction RMSE well below the median components' standard deviation [2.5th quantile - 97.5th quantile] estimated from the examples in the training dataset: 1.400 ns/m [1.358 - 1.441] for X and 4.443 ns [4.255 - 5.183] for Y . Better network configurations should

minimize reconstruction RMSE.

To compare the generated distributions $P_{\tilde{X}}$ and $P_{\tilde{Y}}$ to the marginal priors P_X and P_Y , respectively, we implement a permutation-based kernel two sample test. The test is performed between a sample from the training set of size (the full training size) and an equally sized sample generated randomly by the trained neural network (i.e., new generations for both X and Y , not reconstructions). The procedure of the test is detailed in a subsequent paragraph in this section. Both samples are combined to apply the median heuristic rule for the selection of the correlation length of the Gaussian kernel. We also report median [2.5th quantile - 97.5th quantile] values of biased $\widehat{MMD}_b^2$ estimates.

C2 - Forward approximation We estimate resimulation RMSE in two modes. The first mode is the reconstructions resimulation errors. It applies the forward on reconstructed images X_{recon} and computes RMSE between $F(X_{recon})$ and reconstructed measurement vectors Y_{recon} , without adding any noise to the reconstructions. We can estimate this metric on both training and validation data. The second mode takes randomly generated $\tilde{X}$ and $\tilde{Y}$ (not reconstructed) and computes the RMSE between $\tilde{Y}$ and $F(\tilde{X})$. Both metrics represent how well the network learned to associate specific examples of X and Y together, with the first mode on data similar to the training data and the second on randomly generated data. Both metrics are evaluated on a sample of 2 000 examples, and we report the median [2.5th quantile - 97.5th quantile] estimated on those samples. Similarly to reconstruction errors, we aim for the resimulation errors to be as small as possible. As reference for comparison, we can use the standard deviations of the Y training components 4.443 ns [4.255 - 5.183], and the reconstruction errors previously estimated for Y .

C3 - Latent distribution We perform a kernel two-sample test using a sample of size 2 000 from P_Z and a sample of size 2 000 from Q_Z (once based on training data

and once based on validation data). We report the p-value of the test for each and scaled¹³ median [2.5th quantile - 97.5th quantile] values of biased $\widehat{MMD}_b^2$ estimates. We combined all three latent sets to estimate the median heuristic as the correlation length of the Gaussian kernel.

Kernel two-sample test A kernel two-sample hypothesis test [11] is based on MMD to compare two distributions P and Q using a sample from each. The null hypothesis is that both distributions are the same ($P = Q$), and the alternative is that $P \neq Q$. The test statistic is an unbiased estimate of $\widehat{MMD}_u^2$ based on two samples, one from each distribution being compared.

Given a sample A from P and a sample B from Q of the same size m , with x^A and x^B elements of A and B respectively:

$$\widehat{MMD}_u^2 = \frac{1}{m(m-1)} \sum_{i \neq j} K(x_i^A, x_j^A) - \frac{2}{m^2} \sum_{i,j} K(x_i^A, x_j^B) + \frac{1}{m(m-1)} \sum_{i \neq j} K(x_i^B, x_j^B)$$

with $K(\cdot, \cdot)$ being a characteristic kernel. The sums in $\widehat{MMD}_u^2$ are double sums iterating over all allowed combinations of examples at indices i and j .

We construct the null distribution of the test statistic via simulation and permutation [12] (number of simulations $s = 1000$). In each simulation step, random elements from the samples A and B are exchanged, and $\widehat{MMD}_u^2$ is estimated again on these permuted samples. The p-value is the fraction of simulated $\widehat{MMD}_u^2$ values greater than or equal to the observed test statistic. Our implementation is based on the open-source implementation of [12]¹⁴.

We fix the kernel $K(\cdot, \cdot)$ to the Gaussian kernel as implemented in the scikit-learn

¹³We apply scaling dividing by the median of the similar estimation between samples from the P_Z , to overcome the inflation of the distances due to the latent space dimension

¹⁴https://github.com/emanuele/kernel_two_sample_test/blob/master/kernel_two_sample_test.py

library¹⁵, such that for two vectors x and x' we have:

$$K(x, x') = \exp(-\gamma \|x - x'\|_2^2)$$

with γ set heuristically to the inverse of the median of all pairwise squared Euclidean distances between the examples in the samples being compared during each test.

The Gaussian kernel is characteristic, which makes the MMD a true metric between probability distributions (i.e., $\text{MMD}(P, Q) = 0$ if and only if $P = Q$, for any probability distributions P and Q defined on the same domain).

In our implementation of the test, we estimate γ with large samples from P and Q . For computational reasons, and after the estimation of γ , instead of running one test with all the examples, we perform multiple subtests with subsamples from each of the large samples. Each subtest results in a p-value $\hat{p}_i$, for $i = 1, \dots, T$, yielding T subtests. Each example appears only once in one of the subsamples to ensure that the $\hat{p}_i$ are independent. We combine the p-values with Fisher’s method to construct the statistic $\chi_{2T}^2 = -2 \sum_{i=1}^T \log(\hat{p}_i)$, which follows a chi-square distribution with $2T$ degrees of freedom when the $\hat{p}_i$ are independent and all subtests are assessing the same null hypothesis. From this null chi-square distribution, we assess the probability of observing a value as or more extreme than the observed χ_{2T}^2 and compare it to the 0.05 significance level. This chi-square test effectively assesses whether the null hypothesis “none of the subtests reject their null hypothesis” against the alternative “at least one subtest rejects its null hypothesis”.

A biased estimate MMD is:

$$\widehat{MMD}_b^2 = \frac{1}{m^2} \sum_{i,j} K(x_i^A, x_j^A) - \frac{2}{m^2} \sum_{i,j} K(x_i^A, x_j^B) + \frac{1}{m^2} \sum_{i,j} K(x_i^B, x_j^B)$$

¹⁵https://scikit-learn.org/1.3/modules/generated/sklearn.metrics.pairwise.rbf_kernel.html

Hyperparameters’ tuning results discussion

We discuss our main observations on the jSAE training from the main 36 hyperparameters experiments. Detailed diagnostic metrics of these experiments are provided in the accompanying “linear_exp_hyperparams_results.xlsx” file.

Training routine category We started from the baseline configuration (Exp. ID 101): encoder (no smoothness constraints + PReLU), decoder (constrained PReLU), latent space dimension 30 and standard normal distribution for P_Z , ϵ_{sink} at 100, norms’ power at 2, batch size 200, no β_{sink} scheduling (starting value of 150), no learning rate scheduling, no reconstruction loss scaling, and no data normalization.

Adaptive scaling of β_{sink} from baseline (Exp. ID 102) slightly improved the learning of the prior of X but deteriorated the learning of the prior of Y . Y reconstructions also deteriorated. The resimulation RMSE deteriorated but matching the latent space distributions improved.

When scaling β_{sink} with a fixed factor (Exp. ID 400) (divide β_{sink} by 1.4 every 100 epochs), we deteriorate the learning of both priors, as well as the matching in the latent space, with comparable resimulation and reconstruction RMSE as with adaptive scaling. A particular difficulty here is to select a good fixed scaling factor that maintains the equilibrium between the two parts of the loss for the whole training run. Between the two options of scaling β_{sink} , we maintain the adaptive form to avoid this difficulty.

Compared to the baseline (Exp. ID 101), using Min-Max normalization (Exp. ID 131) considerably improves the learning of both variables priors. Matching the latent space distributions also improved. We retrieve comparable reconstruction and resimulation RMSE as the baseline. We adopt from here on the Min-Max (Exp. ID 131) as the new baseline.

Adding adaptive β_{sink} scaling (Exp. ID 130) to the Min-Max baseline, significantly

improves the learning of the prior of X and greatly deteriorated the learning of the prior of Y (and its reconstructions) as well as the resimulation RMSE. The latent distributions matching only slightly deteriorated.

Maintaining adaptive scaling for β_{sink} and attempting to correct this imbalance in learning Y , we multiply the reconstruction part of Y by a factor that is updated at each epoch (Exp. ID 132). This improved the learning of the prior of Y , but resimulation RMSE did not improve. This behavior was also true without Min-Max normalization (Exp. ID 103).

We think that the adaptive nature of β_{sink} changing during each epoch might be conflicting with the adaptive update of the Y reconstruction error. To stabilize training with adaptive β_{sink} , we apply the *OnPlateau* learning rate scheduling (Exp. ID 1300) (without any adaptation for Y reconstruction). With this configuration we see the biggest improvement across all the metrics. Compared to the *OneCycle* rate scheduling strategy (Exp. ID 1304), the *OnPlateau* remained better on the resimulation RMSE and provided a more stable training overall. The learning rate scheduling seems to allow gradient descent to keep extracting a useful signal for the network weights updates despite the increasingly lower reconstruction estimations and the varying weighting of the latent space divergence at each epoch. In the following, we maintain the Min-Max + adaptive β_{sink} + *OnPlateau* scheduling as the new baseline (Exp. ID 1300).

Scaling the reconstruction part of the loss by the dimension (i.e., divide the reconstruction error of X by 2 000 and the reconstruction error of Y by 80) (Exp. ID 1301) largely deteriorates the learning of the prior of X as well as all reconstructions and resimulation RMSE. We particularly see a large overfit in the reconstruction to the training X data which hints to a form of regularization that was lost by dividing by the dimension. On the other hand, re-multiplying the reconstruction Y part of the loss by a factor updated at each epoch (Exp. ID 1303) deteriorated the learning

of the prior of X with only slight improvement for the reconstruction of Y .

Architecture category We start from the baseline (Exp. ID 1300) Min-Max normalization, adaptive β_{sink} (starting value of 150), *OnPlateau* learning rate scheduling, encoder (no smoothness constraints + PReLU), decoder (constrained PReLU), latent space dimension 30 and standard normal distribution for P_Z , ϵ_{sink} at 100, norms' power at 2, batch size 200, and no reconstruction loss scaling.

Constraining the smoothness of the encoder (spectral normalization + constrained PReLU) (Exp. ID 34) slightly deteriorates the reconstructions, the resimulation and the latent distributions matching.

Using the constrained β -Swish activation in both encoder (without spectral normalization) and decoder (Exp. ID 29) deteriorated the learning of the priors which we think is related to the deterioration of the matching in the latent space (reconstructions and resimulations RMSE remained close to the baseline).

Using β -Swish while constraining the smoothness of the encoder by spectral normalization (Exp. ID 39) improved on the previous with slight deterioration in the reconstruction of Y , the resimulation RMSE and the latent distributions matching.

Varying the latent space dimension has different effects on the metrics (Exp. ID 26, 252, 25, 250, 1300, 251, 27). As expected, the reconstruction RMSE values, on the validation set, decrease with increasing latent space dimension, since we expect that less is lost by the latent space compression. Y reconstruction RMSE show more variability, while X reconstruction RMSE become less variable. The learning of the priors seems to deteriorate as we increase the latent space dimension, mirroring the deterioration of the matching in the latent space, which becomes harder to achieve as the dimension increases. This clearly shows the importance of both parts of the loss in achieving a trade-off between fitting to the training data and generalization. The reconstruction part trains the network to generate samples that are structurally

similar to the training data, while the latent distributions matching part trains the network to distribute its samples in the data space as the prior does: the network can generate samples that were not previously seen in the training but that would still be plausible under the prior. In all latent dimensions tested, the network is able to learn the reconstruction part relatively well, making the latent distributions matching much more relevant in approaching the priors of X and Y . For most dimensions below 35, the quality of matching the latent space distributions is comparable, yet we see a clear gain in reconstruction by increasing dimensions. Furthermore, a ‘sweet spot’ dimension seems to exist, close to 30, where we find an equilibrium between the slightly worse matching in the latent space potentially balanced by the improved reconstructions. Starting dimension 25, little improvement is achieved on the RMSE between the generated $\tilde{Y}$ and the re-simulated generated $\tilde{X}$. This metric in particular hints to the generalization of the network learning of the forward solver, away from the specific training couples. At lower dimensions there is a higher overfitting to training data for X , comparing *RMSE* estimates on training vs validation sets. This indicates that the latent dimension also plays a regularization role.

From this comparison, dimension 30 seems to strike a good balance between matching the priors in terms of structural similarity and distribution in the space while at the same time controlling for overfitting on the training data. We maintain the latent dimension 30 and the current baseline (Exp. ID 1300) for the remaining evaluations.

Loss estimation category We start from the baseline (Exp. ID 1300) Min-Max normalization, adaptive β_{sink} (starting value of 150), *OnPlateau* learning rate scheduling, encoder (no smoothness constraints + PReLU), decoder (constrained PReLU), latent space dimension 30 and standard normal distribution for P_Z , ϵ_{sink} at 100, norms’ power at 2, batch size 200, and no reconstruction loss scaling.

To assess the effect of the batch size, we used the values 100, 200, 300 and 400 (Exp. ID 440, 1300, 44, 45). The same batch size is used for the reconstruction error estimation part and the latent distributions' divergence estimation part of the loss. With higher batch sizes, the estimation of the Sinkhorn divergence is expected to improve. We clearly see this in the improved matching in the latent space with decreasing MMD at batch size 400. However, a higher overfit to X training data is observed along a deterioration of the resimulation errors. The best values for the batch size are 200 or 300, which show very close performances with a very slight deterioration at 300 of reconstructions and resimulations, with an improvement in the latent space matching. Either value could be used forward, we stick with the value 200 as the baseline.

Reducing ϵ_{sink} to 10 (Exp. ID 55), the regularizer of the Sinkhorn divergence, largely deteriorated the matching in the latent space and the priors of the variables. Increasing this regularizer to 1 000 (Exp. ID 67) is not more beneficial than the baseline value of 100. We maintain the value of 100 as the baseline. We note here that the sample size and the ϵ_{sink} value could have a dependent influence on the quality of the estimation of the latent space divergence. We did not proceed to assess this dependence.

When using a power 1 for the norms in the loss (Exp. ID 46), the reconstructions deteriorate, especially Y , as well as the priors.

When testing with a uniform $U(-1, 1)$ as P_Z (Exp. ID 490), we mainly see a deterioration of the latent distribution matching. The network is not able to distribute the embeddings truly uniformly in the latent space and is biased towards concentrating the embeddings around a central location in each dimension. Similar observations can be made with $U(-3, 3)$ (Exp. ID 49) with a more pronounced mismatch between the latent distributions. Using $U(-1, 1)$ and modifying ϵ_{sink} (Exp. ID 610, 490, 730) we reach the same conclusion as with the standard normal that the value 100 remains

the best value tested. Using $U(-1, 1)$ and increasing the batch size to 300 and 400 (Exp. ID 500, 490, 510), makes all metrics comparable to the baseline. We also see an improvement in the latent space matching. However, it is important to note that the network did not actually learn to distribute the embeddings following a $U(-1, 1)$ in the latent space as can be seen from the histograms in Figure 5. Given the shape of these histograms, we think that the embeddings distribution could be close to a multivariate normal centered at the zero vector with a diagonal covariance matrix with diagonal components smaller than 1. To assess this hypothesis, we manually varied the standard deviation of a normal distribution centered on 0, estimated the same MMD metric between samples from these normals and samples from $U(-1, 1)$ and compared those estimates to those observed after network training (from Exp. ID 500). A standard deviation of 0.6 lead to the closest MMD estimates. We trained a network with $\mathcal{N}(0, 0.6^2 I)$ and a batch size of 300 (Exp. ID 80) and it confirmed our intuition since this configuration of the latent space distribution improved on the baseline matching in the latent space. This confirmation of the best latent distribution is necessary because not matching P_Z as closely as possible with Q_Z , does not allow us to confidently use P_Z as a proxy for Q_Z during the inference step. In fact, this evident mismatch with $U(-1, 1)$ (despite the lower metric) increases the risk of sampling from regions in the latent space that are “empty” (where no training data was encoded) resulting in very different, and potentially filled with artifacts, examples compared to the priors P_X and P_Y . Latent distributions mismatch is a common problem in auto-encoder architectures. Some solutions exists, for example, to train another neural network to learn to sample from the embeddings distributions [13], or to restrict the embeddings to map to a specific set of predefined “noise” samples [14]. By finding this parametric configuration for the normal distribution, $\mathcal{N}(0, 0.6^2 I)$, we avoid these extra manipulations and maintain the jSAE formalism all along with a simple sampling mechanism from the latent space.

Finally, we retrain the configuration from Exp. ID 80 with training datasets of size 10 000 (Exp. ID 8010k) and 14 000 (Exp. ID 8014k). Increasing to 10 000 slightly improved X reconstructions while reducing its overfit and slightly improved both resimulation RMSE. Increasing to 14 000 slightly deteriorates Y reconstructions and both resimulation RMSE. We hypothesize that for this architecture, there is no extra gain in increasing the dataset size further than 10 000. The training set size seems to be playing another balancing role between adequately learning each variable and generalization of the forward approximation.

We believe that we have reached the limit of reducing residual errors with this architecture, loss function, and training procedure. We leave investigations to improve on our final baseline to future works. However, an interesting aspect that we wish to explore during the inference step is the robustness of the ABC procedure to these imperfections.

The final configuration that we maintain for the inference step for the linear problem ahead is the following (Exp. ID 8010k): Min-Max normalization, adaptive β_{sink} (starting value of 150), *OnPlateau* learning rate scheduling, encoder (no smoothness constraints + PReLU), decoder (constrained PReLU), latent space dimension 30 and $\mathcal{N}(0, 0.6^2 I)$ distribution for P_Z , ϵ_{sink} at 100, norms' power at 2, batch size 300, and no reconstruction loss scaling, trained with the dataset of size 10 000.

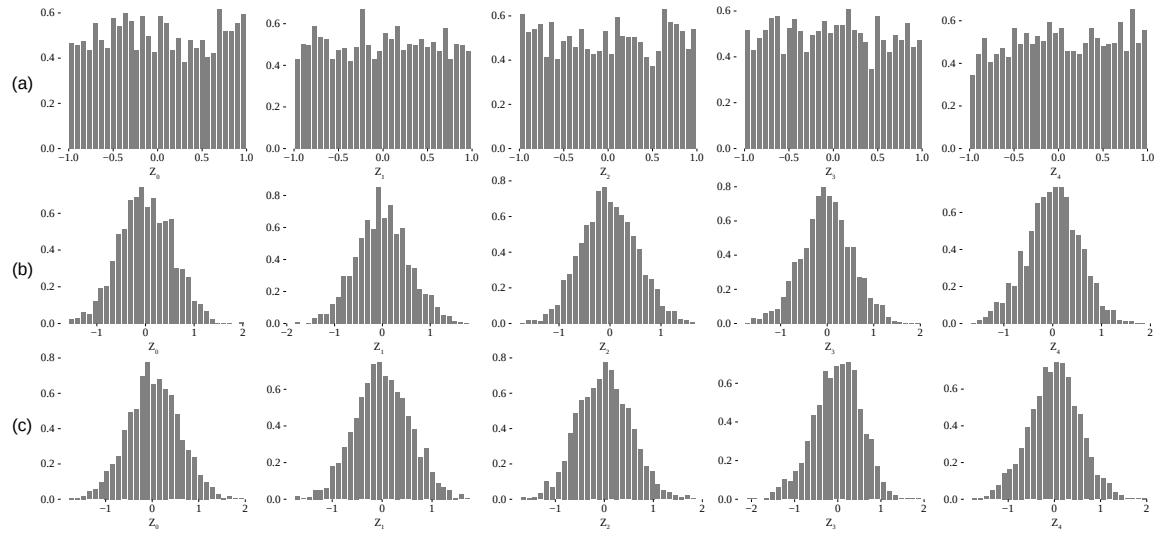


Figure 5: First 5 latent space dimensions histograms: (a) sample from $U(-1, 1)$; (b) training data encoded sample; (c) validation data encoded sample.

Bibliography

- [1] C. Yanagisawa, “Conv2d and convtransposed2d.” https://indico.cern.ch/event/996880/contributions/4188468/attachments/2193001/3706891/ChiakiYanagisawa_20210219_Conv2d_and_ConvTransposed2d.pdf, 2021. Accessed August 24, 2025.
- [2] M. Jia, K. Kumar, L. S. Mackey, A. Putra, C. Vilela, M. J. Wilking, J. Xia, C. Yanagisawa, and K. Yang, “Maximum likelihood reconstruction of water cherenkov events with deep generative neural networks,” *Frontiers in Big Data*, vol. 5, 2022.
- [3] C. White, M. Safari, R. Sukthanker, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter, “Neural architecture search: Insights from 1000 papers,” *arXiv preprint arXiv:2301.08727*, 2023.
- [4] O. Bousquet, S. Gelly, I. Tolstikhin, C.-J. Simon-Gabriel, and B. Schoelkopf, “From optimal transport to generative modeling: the vegan cookbook,” *arXiv preprint arXiv:1705.07642*, 2017.
- [5] P. Rubenstein, B. Schölkopf, and I. Tolstikhin, “Wasserstein auto-encoders: Latent dimensionality and random encoders,” in *International Conference on Learning Representations, ICLR - workshop*, 2018.

- [6] G. Patrini, R. van den Berg, P. Forre, M. Carioni, S. Bhargav, M. Welling, T. Genewein, and F. Nielsen, “Sinkhorn autoencoders,” in *Uncertainty in Artificial Intelligence*, pp. 733–743, PMLR, 2020.
- [7] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” *arXiv preprint arXiv:1710.05941*, 2017.
- [8] P. Ramachandran, B. Zoph, and Q. V. Le, “Swish: A self-gated activation function,” *arXiv preprint arXiv:1710.05941v1*, 2017.
- [9] A. Genevay, L. Chizat, F. Bach, M. Cuturi, and G. Peyré, “Sample complexity of sinkhorn divergences,” in *The 22nd international conference on artificial intelligence and statistics*, pp. 1574–1583, PMLR, 2019.
- [10] A. Genevay, G. Peyre, and M. Cuturi, “Learning generative models with sinkhorn divergences,” in *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics* (A. Storkey and F. Perez-Cruz, eds.), vol. 84 of *Proceedings of Machine Learning Research*, pp. 1608–1617, PMLR, 09–11 Apr 2018.
- [11] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, no. 25, pp. 723–773, 2012.
- [12] E. Olivetti, D. Benozzo, S. M. Kia, M. Ellero, and T. Hartmann, “The kernel two-sample test vs. brain decoding,” *2013 International Workshop on Pattern Recognition in Neuroimaging*, pp. 128–131, 2013.
- [13] A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, and B. Frey, “Adversarial autoencoders,” *arXiv preprint arXiv:1511.05644*, 2015.

- [14] P. Bojanowski and A. Joulin, “Unsupervised learning by predicting noise,” in *International conference on machine learning*, pp. 517–526, PMLR, 2017.